University of Cape Town
Department of Computer Science

Computer Science CSC3003S
ASD Example Exam 2025

Approximate marks per question are shown in brackets.

READ ME FIRST:
This is a sample exam, not an actual exam.
If you are asked for n things and list more than n things, some of which are incorrect, you will receive minus points of any incorrect items. If you list n or fewer things you will not receive minus points for erroneous items.

This sample test has 8 pages.

Time: 2 hours + 10 mins reading time **Marks: 50**

Question 1 [10]

Consider the following pseudocode:

```
void CopyBuffer(int[] dest, int[] source, uint64 length) {
  for (uint64 i = 0; i < length; i++) {
    dest[i] = source[i];
  }
}
```

Assume that your programming language does not protect you; i.e. you are not using a 'safe' language.

a. Name and explain the security problem that lurks in this code. [2]
ASD18 slide 25
Name: buffer overrun/out of bounds write/read [1]
Explain: the function does not check the bounds of dest and source [1]
If length > then either size, then data from other structures will either be read from [1] or overwritten [1]

b. What problem can this security lapse cause? [2]
ASD18 slide 25
Out of bounds reads: exposure of secrets [2]
Out of bounds writes: program crash, execution of arbitrary code, injection of arbitrary data into program [2]

c. Either: rewrite the above code, or describe in prose what changes to make so that the above code is no longer unsecure. *Note that you may need to change the function's signature.* [3]
ASD18 slide 25
`//pass classes that know their own length [1]`

```
//does not have to be const correct, still full points if
//pass by reference & (in vector<int>&) left out.
void CopyBuf2(vector<int>& dest, const vector<int> source) {
  //derive the copy length from the minimum of the two. [1]
  const auto length = min(dest.size(), source.size());
  //set dest length to that minimum length [1]
  //this makes sure you do not leak old dest data [1]
  //alternatively: resize dest to be the size of source [1]
  dest.setsize(length);
  //adding an assert would be good [1]
  assert(length == dest.size() && length <= source.size());
  //copy using the known safe length [1]
  for (uint64 i = 0; i < length; i++) {
    dest[i] = source[i];
  }
}
```

d. Why is using a 'secure' language like Rust (or Java) not enough to fix all security problems? [1]

ASD18+19 all slides

Rust/Java tackle memory issues (out of bounds/use after free/pointer issues) [1]

It does not tackle code injection, man in the middle, encryption issues, authentication etc. etc. [1]

Rust relies on 'unsafe' code for many of the things not allowed by safe Rust, this 'unsafe' sections may have all the memory bugs safe Rust prevents. [1]

e. Rust fixes the security problem shown in points a, b, c.
Which additional security problems does Rust fix?
either describe one (1) problem class, or name two (2) examples. [2]

ASD18+19 all slides

Make sure to scan 1 d. for points to award here.

- class: Multithreading issues [2]
- class: undefined behaviour [2]
- class: memory issues [0 *because that class is covered above*]
- integer overflow [1] (= UB)
- data races [1] (=MT)
- Use after free [1] (=MEM, but different from above)
- Pointer aliasing [1] (=MEM)
- null pointer exceptions [1] (=MEM)

Question 2 [6]

CPUs speed up execution using a number of mutually reinforcing mechanisms. One of these is pipelining.

a. Draw a diagram of a –simplified– CPU instruction pipeline, consisting of a handful of stages, showing multiple instruction cycles. Label the stages of the pipeline and explain briefly what each stage does. [3]

ASD16 slide 10

```
IF   ID   DR   EX   DW
```

```
IF   ID   DR   EX   DW
     IF   ID   DR   EX   DW
          IF   ID   DR   EX   DW
               ... [1]
```

IF: instruction fetch
ID: instruction decode
DR: data read (input parameters)
EX: execute
DW: data write (output) //students may merge DR & DW into a single MEM stage [2]

b. How does this pipelining speed up execution? [1]
ASD16 slide 12
It keeps the different parts of the CPU (IF/ID/...) gainfully employed all the time [1]
It allows the CPU to execute 1 instruction per clock cycle (per pipeline) [1]
By overlapping independent instructions waits are avoided [1]

c. What is out-of-order (OoO) execution and how does it help the pipeline? [2]
ASD16 slide 14+15
The pipeline stalls if instruction B depends on the output of instruction A [1].
In that case you get a 4/5 cycle delay.
By reordering instructions, so that instr. C is executed before B, stalls in the pipeline can be prevented [1]
The CPU uses a output buffer to make sure that memory writes occur in program order (i.e.: outside observers cannot detect the reordering) [1]

Question 3 [8]

a. Xoliswa is a programmer that adheres to the clean coding principles.
Name 3 things that she does in her code
ASD13 slide 2..8 [3]
Meaningful function/variable names [1]
DRY [1]
Low fan-in (classes/functions do not use too many other functions/classes) [1]
SOLID S: Do only one thing in a function/class [1]
Const correctness [1]
No magic constants [1]
No littering of outdated/needless comments [1]

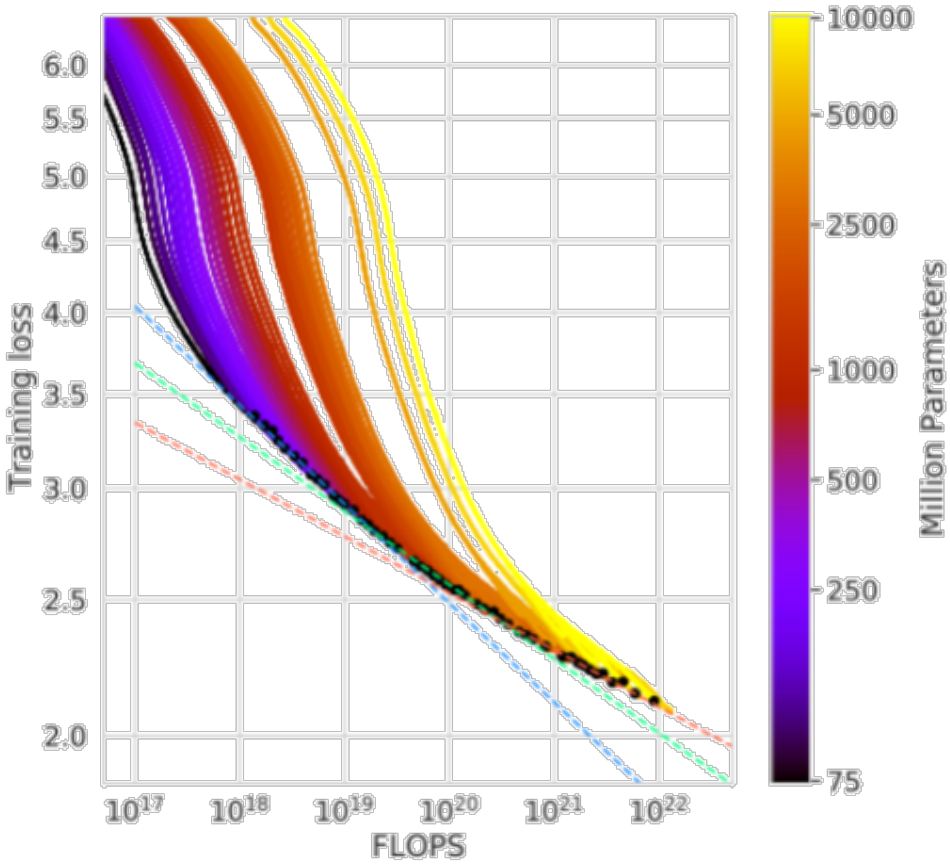
b. Describe the programming workflow used in test driven development [3]
ASD07 slide 21
1. Write code with no/minimal implementation
2. Write test for code => will fail
3. Implement code so (more) tests passes
4. Before adding code: add tests => fail
5. Add more code so tests passes => goto 3 [3]
Q: do we have full coverage?

6. Make tests comprehensive [1]

c. *Xoliswa notices that her non-clean coding colleague also does not use unit tests and argues that such sloppy coding is a violation of professional ethics.*
Put yourself in Xoliswa's shoes and using the ACM/IEEE code of ethics argue her point. [2]
ASD26 slide 6
2.1 High quality [1]
2.2 High standards [1]
2.3 Respect existing rules (i.e.: unit tests not optional) [1]

Question 4 [4]

Consider the following graph detailing AI scaling.
Note that it is a log-log plot, i.e.: both axes scale logarithmically.
This graph shows that training loss (Y-axis) decreases as more FLoating point OPerationS (X-axis) are applied to the training of a generative AI (LLM or otherwise) model. It also shows that models with more parameters follow a different slope w.r.t. FLOPS vs loss. The dotted lines are fits over time (straight lines in this graph, curved lines on a graph with linear axes), blue being the oldest and red the newest fit. Assume that every year sees an increased number of parameters (Y1: black, Y2: purple, Y3: red, Y4: orange, Y5 yellow).



a. *Unsurprisingly, more compute (FLOPS) results in a lower training loss.*
Explain the **real** relationship this graph shows between applied compute power

and training loss in mathematical terms. [2]
ASD20+21 slide 18 right graph
In order to achieve incremental (tiny) improvements in accuracy/training loss [1]
You need exponential increases in compute/FLOPS [1]

Considering that this graph lists actual achieved training losses using applied compute power, what does this graph say about:

b. AI scaling as a function of time? [1]
ASD20 slide 18 left graph
AI scaling has improved **linearly** with time [1]
Decent scaling thus far.
Until the wall will be reached in the near future when compute/electricity supply runs out [1]
Note that the left graph on slide 18 optimistically draws a dotted trendline but that trendline would require 100% (or more) of current worldwide electricity supply (and 100x today's compute, requiring all of worldwide money supply).

c. AI scaling as a function of applied compute power (FLOPS)? [1]
ASD20 slide 18 right graph
The **exponential** slope (the power of the exponent) is getting worse over time [1]
Terrible scaling thus far
Not sustainable because requiring ever more compute power [1]

Question 5 [5]

For the "Pipe and filter architecture pattern" detail the following
ASD22 slide 21 (yellow numbers in top left)

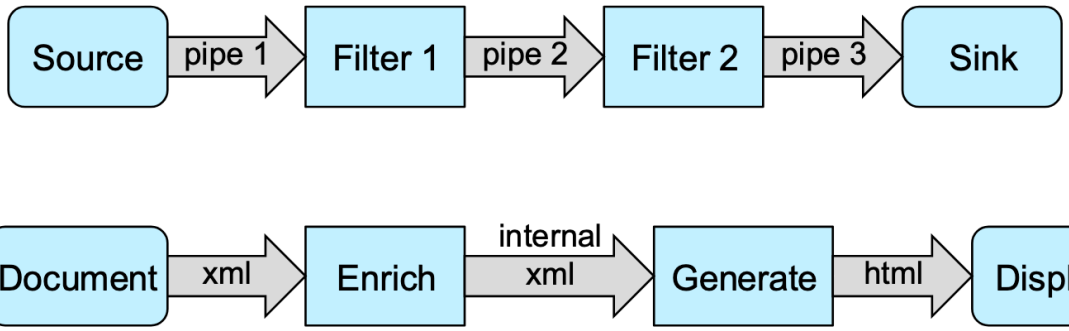
a. Problem (what problem does this pattern solve) [1]
Data processing in stages, input stage B = output stage A [1]

b. Solution (How does it solve it) [1]
Each stage/filter/component is discrete, performs one task
Data flows/pipes from one stage to another [1]

c. Pros/Cons (Benefits + drawbacks using this pattern) [2]
Pros:
Easy to understand + stages can be reused [1]
Workflow/structure matches business processes [1]
Adding stages/filters/... is easy [1]
Can be concurrent or sequential [1]
Cons:
Data must be standardised between stages [1]
Each stages works to this standard [1]
Impossible to reuse stages outside of standard formats [1]
Can be hard to parallelise [1]

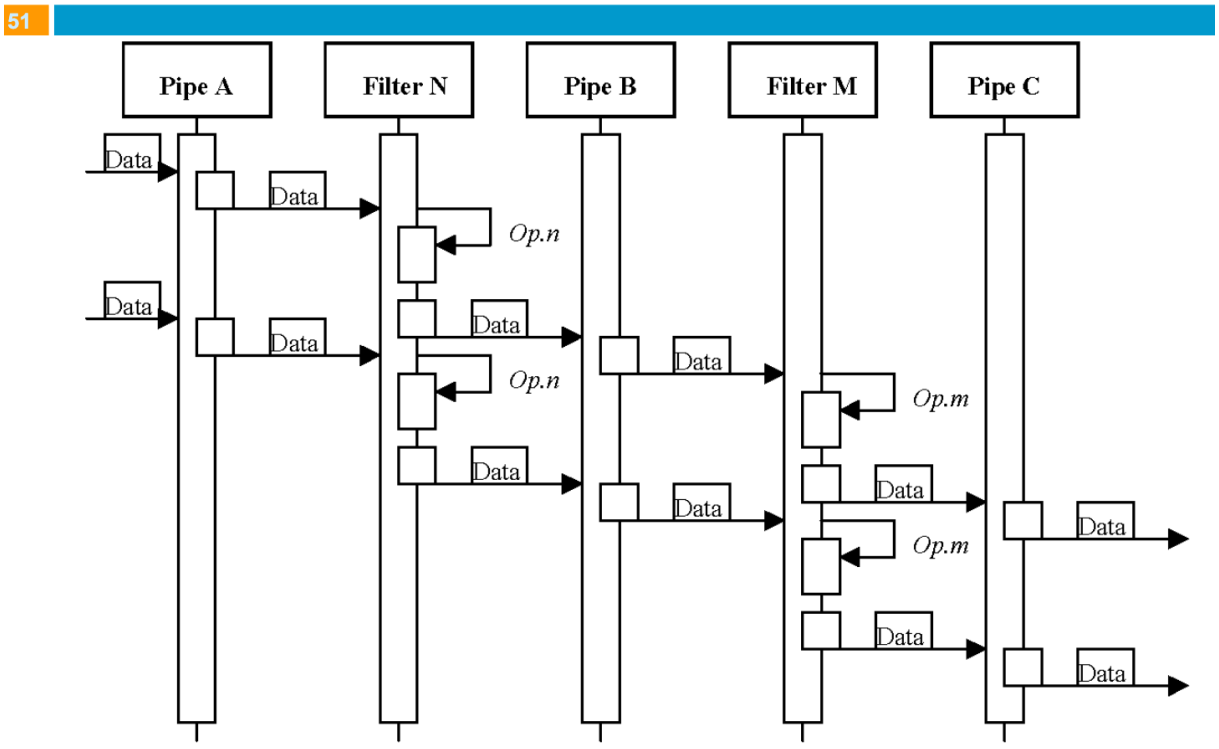
d. Sketch the structure of the chosen pattern using a hard drawn diagram with label as needed. [1]

See ASD 22 slide 22



Or ASD22 slide 51

Parallel Pipe and Filter Diagram



Question 6 [6]

ASD27 slide 12

a. The P in SOAP stands for 'protocol', what do the other letters stand for? [1]

Simple object access protocol [1]

not: Service oriented architecture protocol [0]

b. SOAP is often used in combination with XML
What problem does this cause? [1]
SOAP is a verbose protocol, XML is a verbose data format
=> inefficient communication [1]

c. Would you use SOAP in a micro-services architecture?
Why or why not? [2]
No [1]
Microservices are simple components, SOAP (in spite of its name) is not [1]
Microservices need a lot of communication, favouring a more succinct comms channel [1]

d. How does WSDL (web service description language) relate to SOAP? [2]
It defines the service interface/API [1]
It details what info to decode in the XML communication [1]

Question 7 [6]

Programmer Jane applies the "coding by exception antipattern.
Detail:
ASD 24 slide 18

a. The problem (that this antipattern is erroneously applied to) [1]
Awkward program flow [1]
Validation is already build into exception framework [1]

b. The wrong solution (i.e.: the antipattern itself, the non-fix) [1]
Use exceptions (try/catch/except) to jump to other sections of code (like a goto, or break) [1]
Use exceptions (try/catch/except) as validators [1]

c. Consequence (of implementing the antipattern) [1]
Poor performance (the exception path of exceptions is very slow) [1]
Hard to follow code paths (esp. if exception handler is not visible from exception source) [1]
Accidental hiding of unrelated errors (esp. if using catch-all exceptions) [1]

d. Fix (the correct solution to apply instead) [2]
To escape a deeply nested loop: use a real goto [1]
The exception is a goto, which is considered harmful if overused [1]
Only use exceptions to handle exceptional errors, not for the normal code path [1]
Use case statements/validators/standard solution for your issue instead of exceptions (must match up with abc answers above) [2]

Question 8 [5]

a. Explain the difference between unit testing and integration testing [2]

ASD07 slide 19 + slide 28

Unit testing = test per function **in isolation** [1]

Integration testing = tests of the boundaries **between** modules/classes [1]

b. Complete the following table listing the pros and cons of aspect oriented programming (list 3 pros and 3 cons) [3]

ASD17 slides 15...

Remember only 3 of each, one point per correct pro/con combo, pros and cons need not be opposites

Pros	Cons
Modular (easily add crosscut ops)	Harder to debug, AoP code not visible
Reuse: DRY because AoP code written once	Complexity: crosscuts can be very complex/hard to manually see where they apply
Maintainability: easy to patch existing code	Tooling: IDE may hide crosscut points
SOLID: reduced dependencies, crosscuts separated from main logic	Portability: currently AoP is mostly Java
Scalability: AoP is lightweight	
Testability: DRyer code makes testing easier (if done well, but harder if done poorly)	